Sealing 과정 상세 분석

개요

파일코인의 Sealing(봉인) 과정은 원본 데이터를 암호학적으로 변환하여 저장 증명을 생성할 수 있는 형태로 만드는 핵심 과정입니다. 이 과정은 4개의 주요 단계로 구성되며, 각 단계는 특정한 목적과 기능을 가지고 있습니다.

Sealing 과정의 전체 흐름:

1. **PreCommit Phase 1**: 레이어별 인코딩과 레이블 생성
2. **PreCommit Phase 2**: TreeC, TreeRLast 머클 트리 구축
3. **Commit Phase 1**: 증명 생성을 위한 데이터 준비
4. **Commit Phase 2**: 최종 증명 생성 및 검증

PreCommit Phase 1: 레이어별 인코딩과 레이블 생성

개념과 목적

PreCommit Phase 1은 원본 데이터를 Stacked DRG(Depth Robust Graph) 구조를 사용하여 여러 레이어로 인코딩하는 단계입니다. 이 과정에서 각 노드에 대한 레이블(label)을 생성하고, 이를 통해 데이터의 복제 가능성을 증명할 수 있는 구조를 만듭니다.

주요 구성 요소

1. 입력 데이터 준비

```
// seal_pre_commit_phase1 함수의 핵심 부분

pub fn seal_pre_commit_phase1<R, S, T, Tree: 'static + MerkleTreeTrait>(
```

```
porep_config: &PoRepConfig,
cache_path: R,
in_path: S,
out_path: T,
prover_id: ProverId,
sector_id: SectorId,
ticket: Ticket,
piece_infos: &[PieceInfo],
) -> Result<SealPreCommitPhase1Output<Tree>>
```

입력 매개변수:

- `porep_config` : PoRep 설정 (섹터 크기, 레이어 수 등)
- `cache_path` : 캐시 파일 저장 경로
- `in_path` : 원본 데이터 파일 경로
- `out_path` : 봉인된 데이터 출력 경로
- `prover_id` : 증명자 식별자
- `sector_id` : 섹터 식별자
- `ticket` : 랜덤성 소스
- `piece_infos` : 피스 정보 배열

2. 데이터 복사 및 준비

```
// 원본 데이터를 출력 위치로 복사
if !in_path_is_dev_zero {
```

```
        fs::copy(&in_path, &out_path).with_context(|| {
                format!(
                        "could not copy in_path={:?} to out_path={:?}",
                        in_path.as_ref().display(),
                        out_path.as_ref().display()
                )
        })?;
}

// 파일 크기를 섹터 크기로 확장
f_data.set_len(sector_bytes  as u64)?;
```

3. CommD (Data Commitment) 생성

```
// 원본 데이터에 대한 머클 트리 구축
let  data_tree = create_base_merkle_tree::<BinaryMerkleTree<DefaultPieceHasher>>
        Some(config.clone()),
        base_tree_leafs,
        &data,
)?;

let  comm_d_root: Fr = data_tree.root().into();
```

```
let comm_d = commitment_from_fr(comm_d_root);
```

CommD 생성 과정:

1. 원본 데이터를 32바이트 청크로 분할
2. 각 청크를 해시하여 리프 노드 생성
3. 바이너리 머클 트리 구축
4. 루트 해시를 CommD로 사용

4. Replica ID 생성

```
let replica_id = generate_replica_id::<Tree::Hasher, _>(
        &prover_id,
        sector_id.into(),
        &ticket,
        comm_d,
        &porep_config.porep_id,
);
```

Replica ID 계산:

- `replica_id = Hash(prover_id || sector_id || ticket || comm_d || porep_id)`
- 각 섹터마다 고유한 식별자 생성
- 후속 단계에서 레이블 생성의 시드로 사용

레이어별 레이블 생성 과정

1. Stacked DRG 구조

```
pub struct StackedGraph<H, G> {
    expansion_degree: usize, // 확장 차수 (8)
    base_graph: G, // 기본 DRG 그래프
    feistel_keys: [feistel::Index; 4], // Feistel 암호 키
    feistel_precomputed: FeistelPrecomputed,
    api_version: ApiVersion,
}
```

그래프 특성:

- **Base Degree**: 6 (각 노드가 6개의 기본 부모를 가짐)
- **Expansion Degree**: 8 (각 노드가 8개의 확장 부모를 가짐)
- **Total Degree**: 14 (총 14개의 부모 노드)

2. 레이어 1 레이블 생성

```
// 첫 번째 레이어의 경우
for node in 0..graph.size() {
    create_label(
        graph,
        Some(parents_cache),
        &replica_id,
        &mut layer_labels,
        layer,
        node,
```

```
        )?;
}
```

레이어 1 레이블 계산:

```
label_1[i] = Hash(replica_id || layer || node_id || parents_data)
```

여기서 `parents_data` 는 DRG 그래프에서 정의된 부모 노드들의 데이터입니다.

3. 레이어 2+ 레이블 생성

```
// 두 번째 레이어부터는 확장 부모 포함
for node in 0..graph.size() {
        create_label_exp(
                graph,
                Some(parents_cache),
                &replica_id,
                &exp_labels, // 이전 레이어의 레이블
                &mut layer_labels,
                layer,
                node,
        )?;
}
```

레이어 2+ 레이블 계산:

```
label_n[i] = Hash(replica_id || layer || node_id ||
base_parents_data || exp_parents_data)
```

여기서:

- `base_parents_data` : DRG 기본 부모들의 데이터
- `exp_parents_data` : 이전 레이어에서 선택된 확장 부모들의 레이블

4. 확장 부모 선택 알고리즘

```rust
// Feistel 암호를 사용한 부모 선택
pub fn derive_feistel_keys(porep_id: PoRepID) -> [u64; 4] {
    let mut feistel_keys = [0u64; 4];
    let raw_seed = derive_porep_domain_seed(FEISTEL_DST, porep_id);
    // 32바이트 시드를 4개의 64비트 키로 분할
    feistel_keys[0] = u64::from_le_bytes(raw_seed[0..8].try_into().unwrap())
    feistel_keys[1] = u64::from_le_bytes(raw_seed[8..16].try_into().unwrap()
    feistel_keys[2] = u64::from_le_bytes(raw_seed[16..24].try_into().unwrap(
    feistel_keys[3] = u64::from_le_bytes(raw_seed[24..32].try_into().unwrap(
    feistel_keys
}
```

확장 부모 선택 과정:

1. 현재 노드 인덱스를 Feistel 암호로 변환
2. 변환된 값을 이전 레이어 크기로 모듈로 연산

3. 8개의 서로 다른 확장 부모 선택

4. 선택된 부모들의 레이블을 해시 입력으로 사용

레이어 저장 및 관리

1. 레이어 데이터 저장

```
pub fn write_layer(data: &[u8], config: &StoreConfig) -> Result<()> {
    let data_path = StoreConfig::data_path(&config.path, &config.id);
    let tmp_data_path = data_path.with_extension(".tmp");

    // 임시 파일에 먼저 쓰기
    fs::write(&tmp_data_path, data)?;
    // 원자적 이동으로 최종 저장
    rename(tmp_data_path, data_path)?;
    Ok(())
}
```

2. 부모 캐시 시스템

```
pub struct ParentCache {
    pub path: PathBuf,
    num_cache_entries: u32,
    cache: CacheData,
    pub sector_size: usize,
```

```
        pub  digest: String,

}
```

캐시 최적화:

- 메모리에 부모 노드 정보를 캐시하여 성능 향상
- 디스크 I/O 최소화
- 병렬 처리 지원

Phase 1 출력

```
pub  struct SealPreCommitPhase1Output<Tree> {

        pub  labels: Labels<Tree>, // 생성된 레이블들

        pub  config: StoreConfig, // 저장 설정

        pub  comm_d: Commitment, // 데이터 커밋먼트

}
```

출력 구성 요소:

- **Labels**: 각 레이어의 레이블 저장 위치 정보
- **Config**: 캐시 파일 설정 정보
- **CommD**: 원본 데이터의 머클 트리 루트

PreCommit Phase 2: TreeC, TreeRLast 머클 트리 구축

개념과 목적

PreCommit Phase 2는 Phase 1에서 생성된 레이블들을 사용하여 실제 데이터 인코딩을 수행하고, 두 개의 중요한 머클 트리(TreeC, TreeRLast)를 구축하는 단계입니다.

주요 구성 요소

1. 입력 데이터 로드

```
pub fn seal_pre_commit_phase2<R, S, Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
phase1_output: SealPreCommitPhase1Output<Tree>,
cache_path: S,
replica_path: R,
) -> Result<SealPreCommitOutput>
```

2. 데이터 인코딩 과정

```
// StackedDrg::replicate_phase2 호출
let (tau, (p_aux, t_aux)) = StackedDrg::<Tree, DefaultPieceHasher>::replicate_ph
        &compound_public_params.vanilla_params,
        labels,
        data,
        Some(data_tree),
        cache_path.as_ref().to_path_buf(),
        replica_path.as_ref().to_path_buf(),
)?;
```

데이터 인코딩 메커니즘

1. 레이어별 인코딩

```
pub fn encode<T: Domain>(key: T, value: T) -> T {
    let value: Fr = value.into();
    let mut result: Fr = key.into();
    encode_fr(&mut result, value);
    result.into()
}

pub fn encode_fr(key: &mut Fr, value: Fr) {
    *key += value;
}
```

인코딩 과정:

1. 원본 데이터의 각 노드를 32바이트 요소로 처리
2. 해당 노드의 레이블을 키로 사용
3. `encoded_data[i] = original_data[i] + label[i]` (유한체 연산)
4. 각 레이어마다 순차적으로 인코딩 적용

2. 레이어 간 의존성

```
Layer 1: encoded_1[i] = original[i] + label_1[i]
Layer 2: encoded_2[i] = encoded_1[i] + label_2[i]
```

```
...

Layer N: encoded_N[i] = encoded_{N-1}[i] + label_N[i]
```

TreeC 구축 과정

1. TreeC 개념

TreeC는 각 레이어의 컬럼(column) 해시들로 구성된 머클 트리입니다.

```
// 컬럼 해시 계산
pub fn hash_single_column<H: Hasher>(
column: &Column<H::Domain>,
) -> H::Domain {
        // 컬럼의 모든 요소를 해시
        H::Function::hash(&column.get_data())
}
```

2. 컬럼 구성

```
pub struct Column<T> {
        index: u32,
        rows: Vec<T>,
}
```

컬럼 데이터:

- 각 컬럼은 모든 레이어에서 동일한 위치의 노드들로 구성

- `column[i] = [layer_1[i], layer_2[i], ..., layer_N[i]]`
- 컬럼 해시 = `Hash(layer_1[i] || layer_2[i] || ... || layer_N[i])`

3. TreeC 머클 트리 구축

```
// TreeC 구축 과정
let tree_c = create_disk_tree::<Tree::Hasher>(
        tree_c_size,
        &tree_c_config,
        Some(get_base_tree_count()),
)?;
```

TreeC 구조:

- 리프 노드: 각 컬럼의 해시값
- 내부 노드: 자식 노드들의 해시
- 루트: TreeC의 최종 커밋먼트

TreeRLast 구축 과정

1. TreeRLast 개념

TreeRLast는 마지막 레이어의 인코딩된 데이터로 구성된 머클 트리입니다.

2. 마지막 레이어 데이터 준비

```
// 마지막 레이어의 인코딩된 데이터 사용
let tree_r_last_config = StoreConfig {
        path: cache_path.clone(),
```

```
        id: CacheKey::tree_r_last_tree(layers).to_string(),
        size: Some(get_merkle_tree_len(nodes, BINARY_ARITY)?),
        rows_to_discard: default_rows_to_discard(nodes, BINARY_ARITY),
};
```

3. TreeRLast 머클 트리 구축

```
let  tree_r_last = create_disk_tree::<Tree::Hasher>(
        tree_r_last_size,
        &tree_r_last_config,
        Some(get_base_tree_count()),
)?;
```

CommR 계산

1. CommR 생성 공식

```
// CommR = Hash(CommC || CommRLast)
let  comm_r = <Tree::Hasher as Hasher>::Function::hash2(&comm_c, &comm_r_last);
```

여기서:

- `CommC` : TreeC의 루트 해시
- `CommRLast` : TreeRLast의 루트 해시
- `CommR` : 최종 복제본 커밋먼트

2. Tau 구조체

```
pub struct Tau<T> {
    pub comm_d: T, // 원본 데이터 커밋먼트
    pub comm_r: T, // 복제본 커밋먼트
}
```

보조 데이터 생성

1. PersistentAux (p_aux)

```
pub struct PersistentAux<T> {
    pub comm_c: T,
    pub comm_r_last: T,
}
```

용도:

- 증명 생성 시 필요한 영구 보조 데이터
- TreeC와 TreeRLast의 루트 해시 저장

2. TemporaryAux (t_aux)

```
pub struct TemporaryAux<Tree, G> {
    pub labels: Labels<Tree>,
    pub tree_d_config: StoreConfig,
    pub tree_r_last_config: StoreConfig,
```

```
        pub  tree_c_config: StoreConfig,
        pub  _g: PhantomData<G>,
}
```

용도:

- 증명 생성 시 임시로 필요한 데이터
- 각종 트리와 레이블의 설정 정보 저장

Phase 2 출력

```
pub  struct SealPreCommitOutput {
        pub  comm_r: Commitment, // 복제본 커밋먼트
        pub  comm_d: Commitment, // 데이터 커밋먼트
}
```

Commit Phase 1: 증명 생성을 위한 데이터 준비

개념과 목적

Commit Phase 1은 PreCommit 단계에서 생성된 데이터를 바탕으로 실제 PoRep 증명을 생성하기 위한 준비 작업을 수행하는 단계입니다. 이 단계에서는 challenge를 생성하고, vanilla proof를 만들어 검증합니다.

주요 구성 요소

1. 입력 검증

```
pub fn seal_commit_phase1<T: AsRef<Path>, Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
cache_path: T,
replica_path: T,
prover_id: ProverId,
sector_id: SectorId,
ticket: Ticket,
seed: Ticket, // PoRep challenge 생성 시드
pre_commit: SealPreCommitOutput,
piece_infos: &[PieceInfo],
) -> Result<SealCommitPhase1Output<Tree>>
```

2. 보조 데이터 로드

```
// 영구 보조 데이터 로드
let p_aux = util::get_p_aux::<Tree>(cache_path.as_ref())?;
// 임시 보조 데이터 로드 및 캐시 생성
let t_aux = util::get_t_aux::<Tree>(cache_path.as_ref(),
u64::from(porep_config.sector_size))?;
let t_aux_cache: TemporaryAuxCache<Tree, DefaultPieceHasher> =
TemporaryAuxCache::new(&t_aux, replica_path.as_ref().to_path_buf(), skip_labels)
```

Challenge 생성 과정

1. Public Inputs 구성

```
let public_inputs = stacked::PublicInputs {
        replica_id,
        tau: Some(stacked::Tau {
                comm_d: comm_d_safe,
                comm_r: comm_r_safe,
        }),
        k: None,
        seed: Some(seed),
};
```

2. Private Inputs 구성

```
let private_inputs = stacked::PrivateInputs::<Tree, DefaultPieceHasher> {
        p_aux,
        t_aux: t_aux_cache,
};
```

Vanilla Proof 생성

1. 파티션별 증명 생성

```
let vanilla_proofs = StackedDrg::prove_all_partitions(
        &compound_public_params.vanilla_params,
```

```
        &public_inputs,
        &private_inputs,
        StackedCompound::partition_count(&compound_public_params),
)?;
```

2. Challenge 도출

```
// Interactive challenge 생성
let challenge_positions = interactive_challenges.derive(
        graph.size(),
        &pub_inputs.replica_id,
        &seed,
        k as u8,
);
```

Challenge 생성 알고리즘:

1. `seed` 와 `replica_id` 를 조합하여 랜덤성 생성
2. 섹터 크기 범위 내에서 challenge 위치 선택
3. 각 파티션마다 고유한 challenge 집합 생성

3. 증명 구조

```
pub struct Proof<Tree: MerkleTreeTrait, G: Hasher> {
        pub comm_d_proofs: MerkleProof<G, U2>,
```

```
        pub  comm_r_last_proof: MerkleProof<Tree::Hasher, Tree::Arity>,

        pub  replica_parents: Vec<(u32, DataProof<Tree::Hasher>)>,

        pub  labeling_proofs: Vec<LabelingProof<Tree::Hasher>>,

        pub  encoding_proof: EncodingProof<Tree::Hasher>,

}
```

증명 검증

1. Vanilla Proof 검증

```
let  sanity_check = StackedDrg::<Tree, DefaultPieceHasher>::verify_all_partition

        &compound_public_params.vanilla_params,

        &public_inputs,

        &vanilla_proofs,

)?;

ensure!(sanity_check, "Invalid vanilla proof generated");
```

2. 검증 과정

1. **CommR 검증**: 계산된 CommR이 예상값과 일치하는지 확인
2. **Challenge 검증**: 각 challenge에 대한 증명이 유효한지 확인
3. **레이블링 증명**: 레이블 생성이 올바른지 검증
4. **인코딩 증명**: 데이터 인코딩이 정확한지 검증

Phase 1 출력

```
pub struct SealCommitPhase1Output<Tree> {
    pub vanilla_proofs: Vec<Vec<Proof<Tree, DefaultPieceHasher>>>,
    pub comm_r: Commitment,
    pub comm_d: Commitment,
    pub replica_id: <Tree::Hasher as Hasher>::Domain,
    pub seed: Ticket,
    pub ticket: Ticket,
}
```

Commit Phase 2: 최종 증명 생성 및 검증

개념과 목적

Commit Phase 2는 Phase 1에서 생성된 vanilla proof들을 zk-SNARK 회로 증명으로 변환하여 최종적으로 검증 가능한 간결한 증명을 생성하는 단계입니다.

주요 구성 요소

1. Circuit Proof 생성

```
pub fn seal_commit_phase2_circuit_proofs<Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
phase1_output: SealCommitPhase1Output<Tree>,
sector_id: SectorId,
) -> Result<SealCommitOutput>
```

2. Groth16 증명 생성

```
let  groth_proofs = StackedCompound::<Tree, DefaultPieceHasher>::circuit_proofs(
        &public_inputs,
        vanilla_proofs,
        &compound_public_params.vanilla_params,
        &groth_params,
        compound_public_params.priority,
)?;
```

zk-SNARK 회로 구성

1. 회로 입력

- **Public Inputs**: CommD, CommR, replica_id, seed
- **Private Inputs**: vanilla proofs, 보조 데이터

2. 회로 제약 조건

1. **머클 트리 검증**: TreeC, TreeRLast의 일관성 확인
2. **레이블 생성 검증**: 각 레이어의 레이블이 올바르게 생성되었는지 확인
3. **인코딩 검증**: 데이터 인코딩이 정확한지 확인
4. **Challenge 응답 검증**: 선택된 challenge에 대한 응답이 유효한지 확인

증명 집계 (Aggregation)

1. 단일 증명 vs 집계 증명

```
// Non-interactive PoRep의 경우 집계 증명 생성
let buf = if porep_config.feature_enabled(ApiFeature::NonInteractivePoRep) {
        aggregate_seal_commit_proofs::<Tree>(
                porep_config,
                &[comm_r],
                &[seed],
                &[seal_commit_output],
                groth16::aggregate::AggregateVersion::V2,
        )?
} else {
        seal_commit_output.proof
};
```

2. 집계 증명 생성 과정

```
pub fn aggregate_seal_commit_proofs<Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
comm_rs: &[[u8; 32]],
seeds: &[[u8; 32]],
commit_outputs: &[SealCommitOutput],
aggregate_version: groth16::aggregate::AggregateVersion,
) -> Result<AggregateSnarkProof>
```

집계 과정:

1. 여러 개의 개별 증명을 수집
2. SnarkPack 알고리즘을 사용하여 단일 증명으로 집계
3. 검증 시간과 저장 공간 최적화

최종 검증

1. 사후 검증 (Post-seal Verification)

```
let  is_valid = verify_seal::<Tree>(
        porep_config,
        comm_r,
        comm_d,
        prover_id,
        sector_id,
        ticket,
        seed,
        &buf,
)?;
ensure!(is_valid, "post seal aggregation verifies");
```

2. 검증 과정

1. **증명 파싱**: 바이트 배열에서 증명 구조체로 변환
2. **Public Input 재구성**: 검증에 필요한 공개 입력 생성
3. **Groth16 검증**: zk-SNARK 증명의 수학적 유효성 확인

4. **커밋먼트 일관성**: CommD, CommR의 일관성 확인

Phase 2 출력

```rust
pub struct SealCommitOutput {
    pub proof: Vec<u8>, // 최종 증명 바이트 배열
}
```

성능 최적화 기법

1. 병렬 처리

- **멀티코어 SDR**: 여러 CPU 코어를 활용한 레이블 생성
- **GPU 가속**: 머클 트리 구축 시 GPU 활용
- **파티션 병렬화**: 증명 생성을 여러 파티션으로 분할

2. 메모리 최적화

- **부모 캐시**: 자주 사용되는 부모 노드 정보 캐싱
- **스트리밍 처리**: 대용량 데이터를 청크 단위로 처리
- **메모리 매핑**: 파일 I/O 최적화

3. 디스크 I/O 최적화

- **원자적 쓰기**: 임시 파일 사용으로 데이터 무결성 보장
- **압축**: 중간 데이터의 압축 저장
- **캐시 계층화**: 메모리-디스크 캐시 계층 구조

보안 고려사항

1. 시간-공간 트레이드오프 방어

- **Sequential Dependency**: 레이어 간 순차적 의존성으로 병렬화 방지
- **Memory Hard**: 대용량 메모리 요구로 하드웨어 공격 방어

2. 암호학적 보안

- **Feistel 암호**: 부모 선택의 의사랜덤성 보장
- **해시 함수 다양성**: SHA256, Blake2s, Poseidon 등 다양한 해시 함수 사용

3. 검증 가능성

- **zk-SNARK**: 영지식 증명으로 프라이버시 보장
- **Public Verifiability**: 누구나 증명을 검증할 수 있는 구조

결론

파일코인의 Sealing 과정은 복잡하지만 체계적으로 설계된 4단계 프로세스입니다. 각 단계는 특정한 목적을 가지고 있으며, 전체적으로 안전하고 효율적인 저장 증명 시스템을 구현합니다. 이 과정을 통해 생성된 증명은 파일코인 네트워크에서 저장 제공자가 실제로 데이터를 저장하고 있음을 증명하는 핵심 메커니즘으로 작동합니다.